LINUX SYSTEM SECURITY

Hands-On Laboratory Report

Labs 1 – 12

Sancho Macbryan

Matriculation Number: Uba25EP107

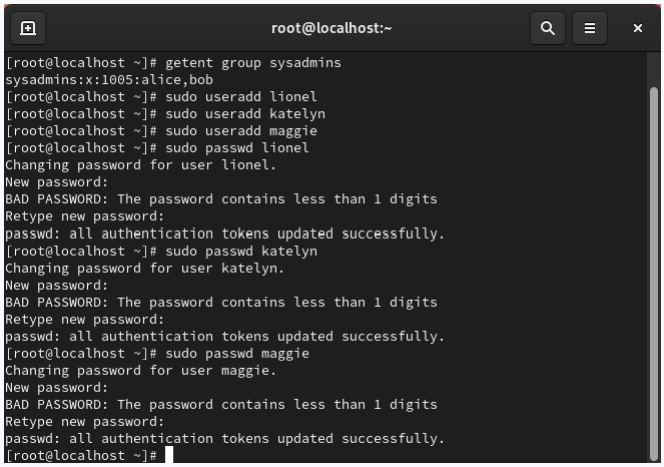
1. Assigning Limited Sudo Privileges

Objective: Create user accounts and assign them different levels of sudo privileges on AlmaLinux 9.

Step 1 Create User Accounts

Log in to the AlmaLinux virtual machine and create user accounts for Lionel, Katelyn, and Maggie, then set their passwords:

sudo useradd lionel
sudo useradd katelyn
sudo useradd Maggie
sudo passwd lionel
sudo passwd katelyn
sudo passwd Maggie

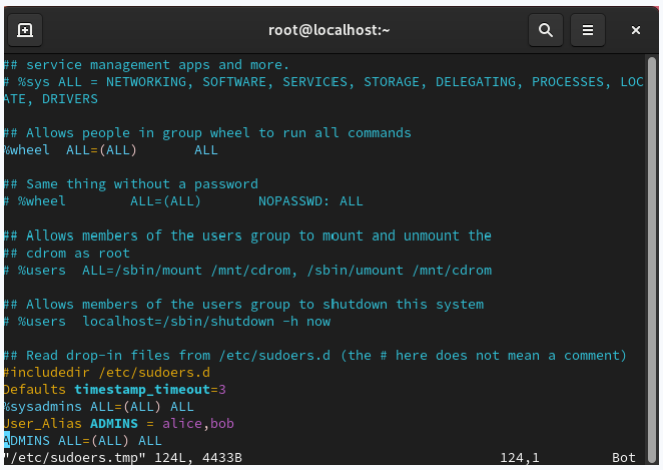


Screenshot: User accounts created useradd and passwd commands output

Step 2 Open the Sudoers File with visudo

Open the sudoers file for safe editing using the visudo command:

sudo visudo



Screenshot: visudo editor open showing sudoers file

Step 3 Uncomment the STORAGE Command Alias

Inside visudo, find the STORAGE command alias section and remove the comment symbol (#) from in front of it to activate it.

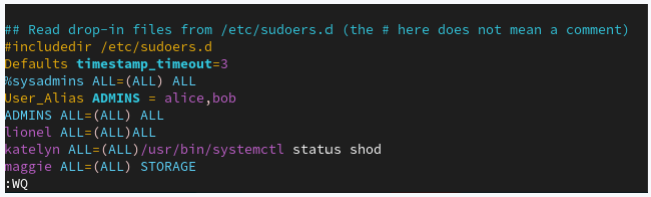
Screenshot: STORAGE alias uncommented in visudo

Step 4 Add Privilege Entries for Each User

At the end of the sudoers file, add the following lines (use tabs to separate the columns):

lionel ALL=(ALL) ALL
katelyn ALL=(ALL) /usr/bin/systemctl status shod
maggie ALL=(ALL) STORAGE

Save the file and exit visudo.



Screenshot: New lines added at the end of sudoers file

Step 5 Test Lionel's Full Sudo Privileges

Switch to Lionel's account and verify that he has full sudo privileges by running several root-level commands:

su - lionel

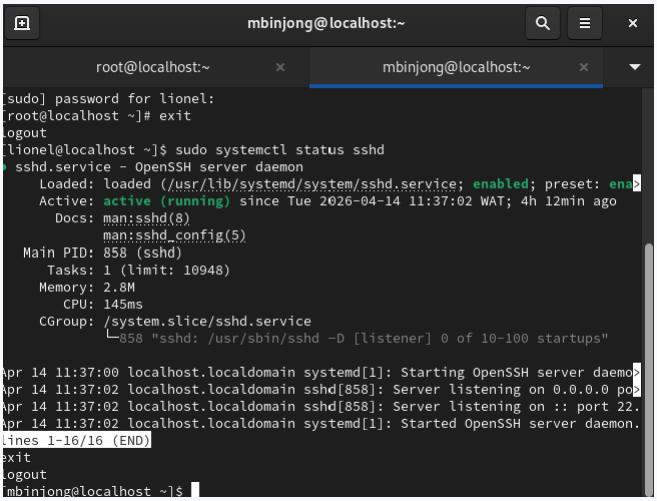
sudo su -

exit

sudo systemctl status sshd

sudo fdisk -l

exit

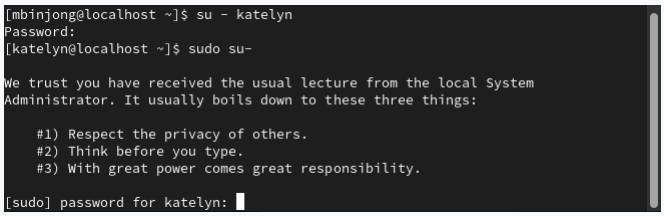


Screenshot: Lionel successfully running root-level commands (sudo su -, systemctl, fdisk)

Step 6 Test Katelyn's Limited Privileges

Log in as Katelyn and attempt to run various root-level commands to observe which are allowed and which are denied:

su - katelyn
sudo su -
sudo systemctl status sshd
sudo systemctl restart sshd
sudo fdisk -l
exit

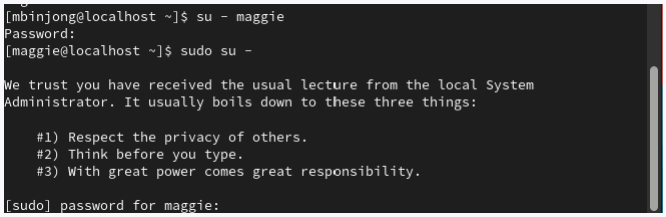


Screenshot: Katelyn allowed for 'systemctl status sshd' but denied for 'sudo su -' and 'fdisk'

Step 7 Test Maggie's STORAGE Privileges

Log in as Maggie and run the same set of commands that were run for Katelyn. Observe that Maggie is only permitted to use STORAGE-related commands:

su - maggie
sudo su -
sudo systemctl status sshd
sudo systemctl restart sshd
sudo fdisk -l
exit



Screenshot: Maggie's command results – restricted to STORAGE alias commands only

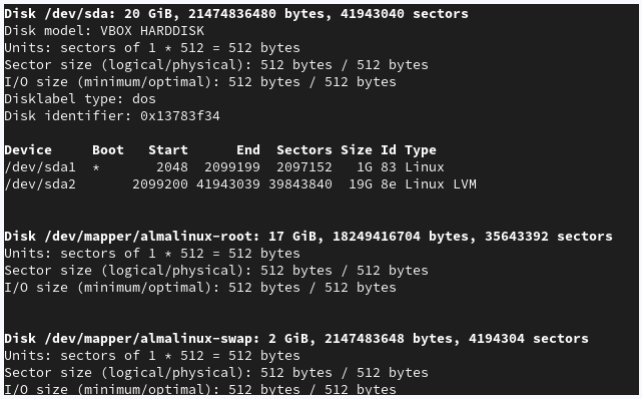
2. Disabling sudo timer.

Objective: Disable the sudo timer so that the system requires a password for every sudo command.

Step 1 Run Initial Sudo Commands

Run the following commands and observe that after the first password entry, subsequent commands do not require re-authentication:

```
sudo fdisk -l
sudo systemctl status sshd
sudo iptables -L
```



```
Disk /dev/sda: 20 GiB, 21474836480 bytes, 41943040 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x13783f34

Device     Boot   Start      End  Sectors Size Id Type
/dev/sda1  *       2048  2099199  2097152   1G 83 Linux
/dev/sda2       2099200 41943039 39843840  19G 8e Linux LVM


Disk /dev/mapper/almalinux-root: 17 GiB, 18249416704 bytes, 35643392 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/mapper/almalinux-swap: 2 GiB, 2147483648 bytes, 4194304 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

Screenshot: Commands run without re-entering password (sudo timer active)

Step 2 Reset the Sudo Timer with sudo -k

Run the following commands and note how sudo -k resets the timer, requiring you to re-enter your password:

```
sudo fdisk -l
sudo -k
sudo fdisk -l
```

```
Disk /dev/sda: 20 GiB, 21474836480 bytes, 41943040 sectors
Disk model: VBOX HARDDISK
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x13783f34

Device     Boot   Start      End  Sectors Size Id Type
/dev/sda1  *       2048  2099199  2097152   1G 83 Linux
/dev/sda2       2099200 41943039 39843840  19G 8e Linux LVM


Disk /dev/mapper/almalinux-root: 17 GiB, 18249416704 bytes, 35643392 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes


Disk /dev/mapper/almalinux-swap: 2 GiB, 2147483648 bytes, 4194304 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
```

Screenshot: sudo -k forces password re-entry for the next sudo command

Step 3 Disable the Timer Globally in visudo

Open visudo and add the following line in the Defaults specification section:

sudo visudo

Add the line below to the Defaults section:

Defaults timestamp timeout = 0

Save the file and exit visudo.

```
## Read drop-in files from /etc/sudoers.d (the # here does not mean a comment)
#includedir /etc/sudoers.d
Defaults timestamp_timeout=0
```

Screenshot: visudo with 'Defaults timestamp timeout = 0 added

Step 4 Verify Global Timer Disabled

Run the commands from Step 1 again. This time, a password should be required for every sudo command:

sudo fdisk -l

sudo systemctl status sshd

sudo iptables -L

```
I/O size (minimum/optimal): 512 bytes / 512 bytes
[maggie@localhost ~]$ sudo fdisk -l
[sudo] password for maggie:
```

Screenshot: Every sudo command now prompts for a password

Step 5 Apply Timer Restriction to Lionel Only

Open visudo and modify the line added in Step 3 so it targets only Lionel's account:

```
sudo visudo

Defaults: Lionel timestamp timeout = 0
```

Save and exit visudo.

```
#includedir /etc/sudoers.d
Defaults:lionel timestamp_timeout=0
```

Screenshot: visudo updated with Lionel-specific timestamp timeout setting

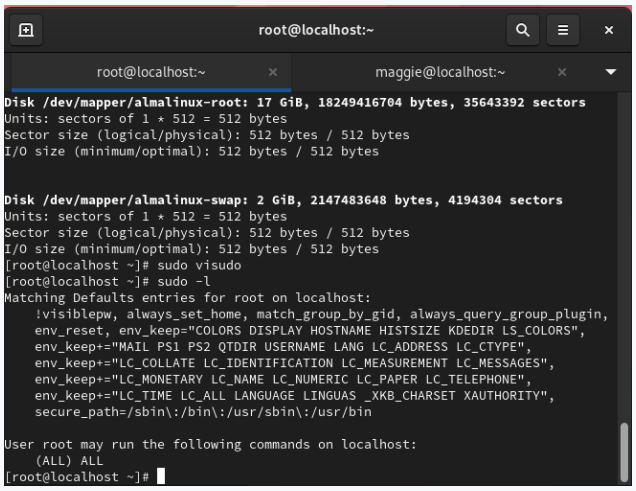
Step 6 Verify Per-User Timer and View Privileges

From your own account, run the sudo commands (you should now have a timer). Then log in as Lionel and repeat (Lionel always needs a password):

```
sudo fdisk -l
sudo systemctl status sshd
su - lionel
sudo systemctl status sshd
exit
```

View your own sudo privileges:

```
sudo -l
```



Screenshot: sudo -l output showing current user's sudo privileges

3. LAB 3 Creating an Encrypted Home Directory with adduser

Objective: Create a user account with an encrypted home directory using ecryptfs on Ubuntu 22.04.

Step 1 Install ecryptfs-utils Package

1. **Enable the EPEL repository:**
   sudo dnf install epel-release -y
2. **Install the utilities:**
   sudo dnf install ecryptfs-utils -y

Step 2 Create User Cleopatra with Encrypted Home Directory

sudo adduser --encrypt-home cleopatra

Follow the prompts to set a password for Cleopatra and provide any required user information.

Step 3 View the Home Directory Listing

ls -l /home

Step 4 Log in as Cleopatra and Retrieve Passphrase

Switch to Cleopatra's account and run the ecryptfs-unwrap-passphrase command to view the encrypted passphrase:

```
su - cleopatra
ecryptfs-unwrap-passphrase
exit
```

4. LAB 4 Setting Password Complexity Criteria

Objective: Configure password quality requirements using the pwquality module on AlmaLinux 9.

Step 1 Open pwquality.conf and Set Minimum Length

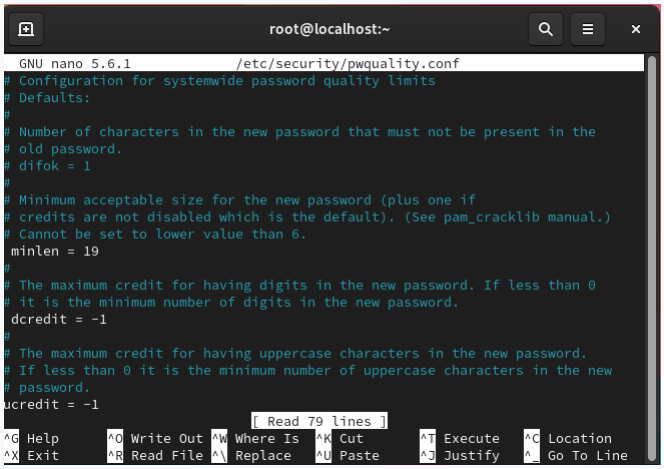
Open the /etc/security/pwquality.conf file in your preferred text editor:

```
sudo nano /etc/security/pwquality.conf
```

Find the minlen line, remove the comment symbol, and change the value to 19:

```
minlen = 19
```

Save the file and exit the editor.



Screenshot: pwquality.conf with minlen = 19 uncommented and set

Step 2 Create User Goldie and Test Short Passwords

Create a user account for Goldie and attempt to assign the following passwords, observing the warning messages for each:

```
sudo useradd goldie
sudo passwd goldie
```

Try assigning the following passwords one at a time and note the error messages:

- turkeylips (all lowercase, too short)
- TurkeyLips (mixed case, too short)

- Turkey93Lips (meets more criteria but still too short)

```
[root@localhost ~]# sudo useradd goldie
[root@localhost ~]# sudo passwd goldie
Changing password for user goldie.
New password:
BAD PASSWORD: The password contains less than 1 digits
Retype new password:
passwd: all authentication tokens updated successfully.
[root@localhost ~]#
```

Screenshot: Password rejection messages for 'turkeylips', 'TurkeyLips', and 'Turkey93Lips'

Step 3 Update pwquality.conf with Class and Repeat Rules

In pwquality.conf, comment out the minlen line. Uncomment minclass and maxclassrepeat, and set maxclassrepeat to 5:

```
# minlen = 19
minclass = 3
maxclassrepeat = 5
```

Save the file and exit.

```
#
# The minimum number of required classes of characters for the new
# password (digits, uppercase, lowercase, others).
 minclass = 3
#
# The maximum number of allowed consecutive same characters in the new password.
# The check is disabled if the value is 0.
# maxrepeat = 0
#
# The maximum number of allowed consecutive characters of the same class in the
# new password.
# The check is disabled if the value is 0.
 maxclassrepeat = 5
#
# Whether to check for the words from the passwd entry GECOS string of the user.
# The check is enabled if the value is not 0.
```

Screenshot: pwquality.conf updated with minclass = 3 and maxclassrepeat = 5

Step 4 Test the New Complexity Rules

Try assigning various non-compliant passwords to Goldie's account and observe the rejection messages:

```
sudo passwd goldie
```

```
root@localhost ~]# sudo passwd goldie
hanging password for user goldie.
ew password:
AD PASSWORD: The password contains less than 1 digits
etype new password:
```

Screenshot: Various password complexity rejection messages with the new rules active

5. LAB 5 Setting Account and Password Expiry Data

Objective: Manage account expiry dates and password aging policies using useradd, usermod, and chage on AlmaLinux 9.

Step 1 Create User Samson with an Expiration Date

Create a user account for Samson with an expiration date of June 30, 2025, and view the account details:

sudo useradd -e 2025-06-30 samson

sudo chage -l samson

```
[root@localhost ~]# sudo useradd -e 2025-06-30 samson
[root@localhost ~]# sudo chage -l samson
Last password change                                    : Apr 15, 2026
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : Jun 30, 2025
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
[root@localhost ~]#
```

Screenshot: Samson's account created with expiry 2025-06-30; chage -l output

Step 2 Change Samson's Account Expiration Date with usermod

Use usermod to update Samson's account expiration date to July 31, 2025:

sudo usermod -e 2025-07-31 samson

sudo chage -l samson

```
[root@localhost ~]# sudo usermod -e 2025-07-31 samson
[root@localhost ~]# sudo chage -l samson
Last password change                                    : Apr 15, 2026
Password expires                                        : never
Password inactive                                       : never
Account expires                                         : Jul 31, 2025
Minimum number of days between password change          : 0
Maximum number of days between password change          : 99999
Number of days of warning before password expires       : 7
[root@localhost ~]#
```

Screenshot: chage -l showing updated expiry date of 2025-07-31 for Samson

Step 3 Assign Password and Force Reset on First Login

Assign a password to Samson and force him to change it on his first login:

```
sudo passwd samson
sudo passwd -e samson
sudo chage -l samson
su - samson
exit
```

Step 4 Configure Full Password Aging Policy with chage

Use chage to set a comprehensive password aging policy for Samson:

- Minimum 5 days before password can be changed (-m 5)
- Password expires after 90 days (-M 90)
- Account locked after 2 days of inactivity (-I 2)
- Warning issued 5 days before expiry (-W 5)

```
sudo chage -m 5 -M 90 -I 2 -W 5 samson
sudo chage -l samson
```

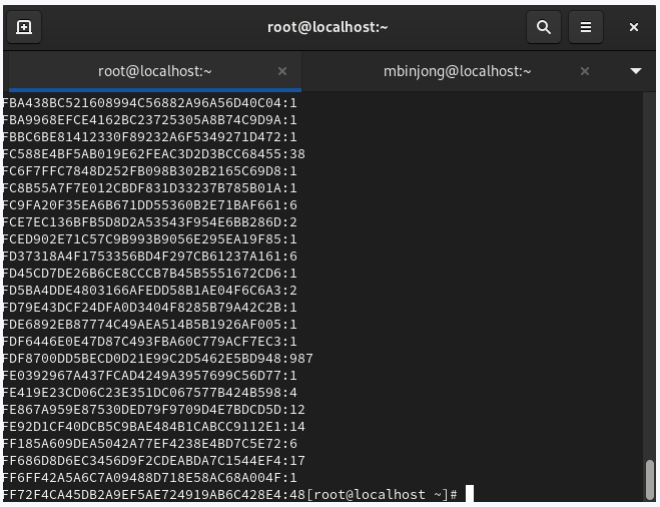
6. LAB 6 Detecting Compromised Passwords

Objective: Use the Have I Been Pwned API and a shell script to check whether passwords have appeared in known data breaches.

Step 1 Query the Pwned Passwords API with curl

Use curl to query the API using the first 5 characters of a SHA-1 hash prefix and observe the results:

```
curl https://api.pwnedpasswords.com/range/21BD1
```



Screenshot: curl output showing list of breached password hash suffixes with occurrence counts

Step 2 Obtain the pwnedpasswords.sh Script

In the home directory of your AlmaLinux VM, paste the pwnedpasswords.sh script provided by the lecturer. The script hashes a password locally, sends only the first 5 characters of the hash to the API, and then checks the returned list without ever sending the full password.

Step 3 Make the Script Executable

```
chmod u+x pwnedpasswords.sh
```

Step 4 Test the Script with a Known Compromised Password

Run the script with 'TurkeyLips' as the test password:

```
./pwnedpasswords.sh TurkeyLips
```

Step 5 Test with Additional Passwords

Repeat the test with different passwords of your choice to compare results:

```
./pwnedpasswords.sh <your_test_password_1>
./pwnedpasswords.sh <your_test_password_2>
```

7. LAB 7 Configuring pam_tally2

Objective: Configure the pam_tally2 PAM module to lock user accounts after repeated failed login attempts (CentOS 7 / AlmaLinux).

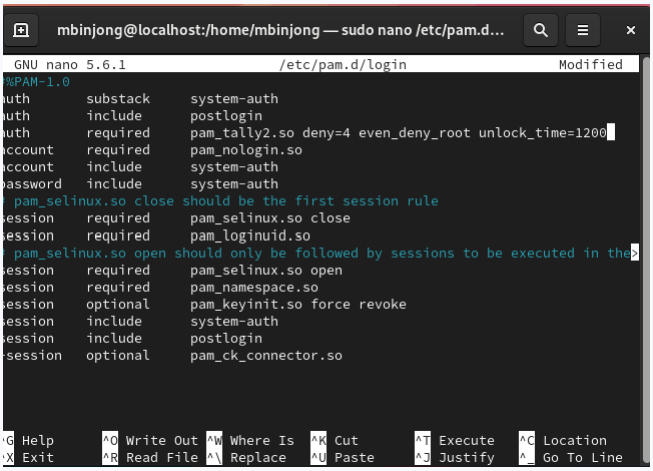
Step 1 Configure /etc/pam. d/login

Open the /etc/pam. d/login file for editing. Locate the line that invokes pam_securetty (around line 2) and insert the following line directly beneath it:

```
sudo nano /etc/pam. d/login
```

```
auth required pam_tally2.so deny=4 even_deny_root unlock_time=1200
```

Save the file and exit.

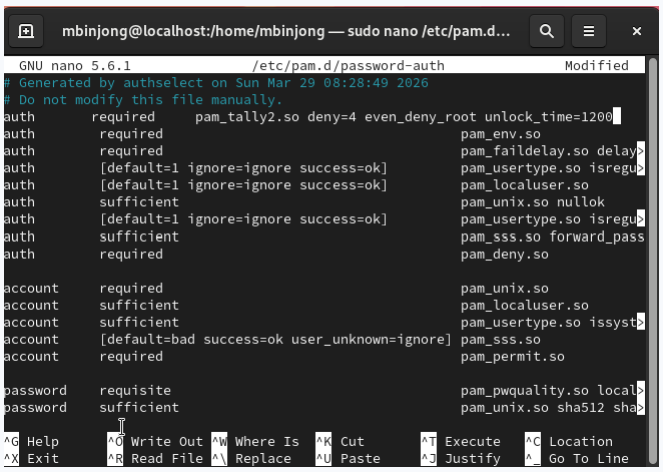


Screenshot: /etc/pam. d/login with pam_tally2 line inserted below pam_securetty

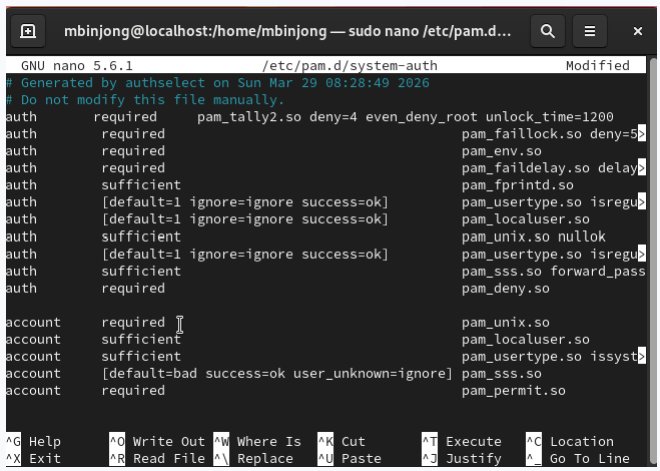
Step 2 – Configure password-auth and system-auth

Place the same pam_tally2 line at the top of both /etc/pam.d/password-auth and /etc/pam.d/system-auth, just above the first 'auth required' line:

sudo nano /etc/pam. d/password-auth
sudo nano /etc/pam. d/system-auth



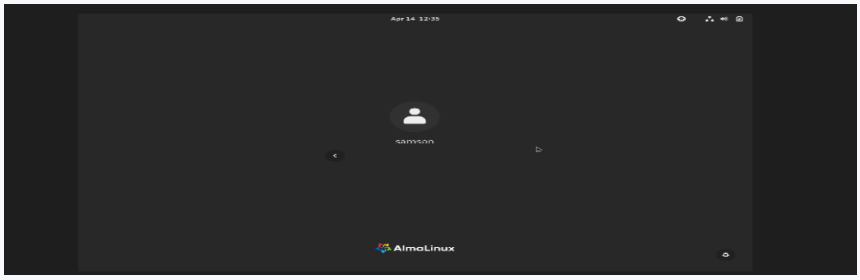
Screenshot: /etc/pam.d/password-auth with pam_tally2 line at the top



Screenshot: /etc/pam.d/system-auth with pam_tally2 line at the top

Step 3 Trigger Account Lockout for Samson

Log out of your account. At the login screen, deliberately enter the wrong password for ur account five times until the account lockout message appears:



Screenshot: Login screen showing account lockout message for Samson after 5 failed attempts

Step 4 pam_tally2 Counters

Log back in to your own user account and check the tally counter:

```
sudo pam_tally2
```

Step 5 Unlock Samson's Account

Simulate a help desk scenario: verify the caller's identity, then reset Samson's tally counter to unlock the account:

```
sudo pam_tally2 --user=samson --reset
sudo pam_tally2
```

Step 6 Update deny Value to a More Realistic Threshold

Edit /etc/pam. d/login and change the deny parameter from 4 to 100 for a more modern security policy:

```
sudo nano /etc/pam. d/login
```

```
auth required pam_tally2.so deny=100 even_deny_root unlock_time=1200
```

8. LAB 8 Distributed Identity Management

Objective: Install and configure distributed identity management tools: FreeIPA and Adsys.

Part A FreeIPA Installation and Configuration

FreeIPA is an integrated identity and authentication solution for Linux/UNIX environments. It combines LDAP, Kerberos, DNS, and certificate management.

Step 1 Install FreeIPA Server

On AlmaLinux 9, install the FreeIPA server package:

```
sudo dnf install freeipa-server freeipa-server-dns
```

Step 2 Run the FreeIPA Setup Wizard

Run the ipa-server-install command and follow the interactive prompts to configure the realm, domain, and administrator passwords:

```
sudo ipa-server-install
```

Step 3 Enable and Start FreeIPA Services

```
sudo systemctl enable --now ipa
```

Part B Adsys Installation and Configuration

Adsys is a tool for integrating Ubuntu machines into Active Directory environments with Group Policy support.

Step 4 Install Adsys (Ubuntu)

```
sudo apt install adsys
```

Step 5 Configure and Verify Adsys

Configure Adsys to connect to your Active Directory domain and verify the connection:

```
sudo adsysctl policy update
```

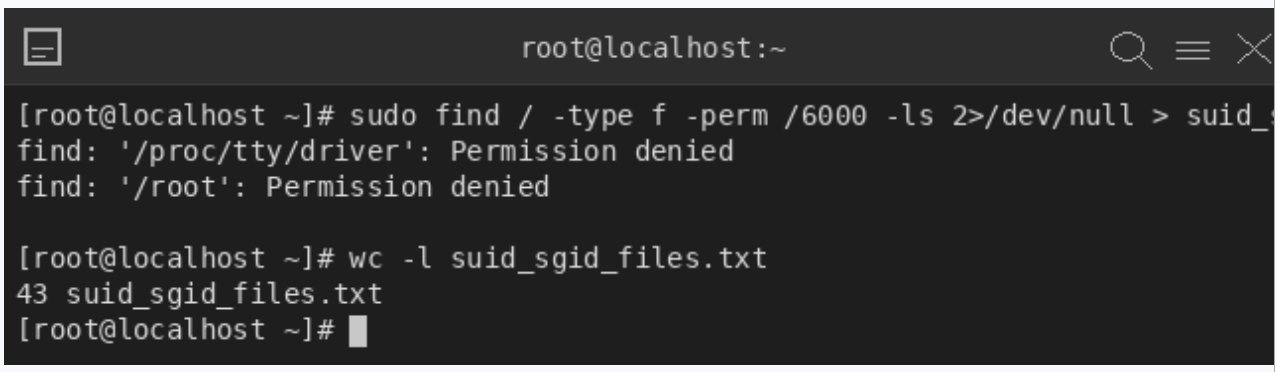
9. LAB 9 Searching for SUID and SGID Files

Objective: Use the find command to locate SUID and SGID files on the system, then compare the results before and after creating a new SUID file.

Step 1 Search Filesystem for SUID/SGID Files

Search the entire filesystem for all files with SUID or SGID bits set and save the output to a text file:

```
sudo find / -type f -perm /6000 -ls > suid_sgid_files.txt
```



Screenshot: find command executing and output redirected to suid_sgid_files.txt

Step 2 Create a Dummy SUID Script as Another User

Log in to another user account, create a shell script, set the SUID permission on it, verify it, then log back out:

```
su - lionel
touch some_shell_script.sh
chmod 4755 some_shell_script.sh
ls -l some_shell_script.sh
exit
```

```
root@localhost:~
[root@localhost ~]# head -20 suid_sgid_files.txt
  131079    68 -rwsr-xr-x   1 root     root       68768 Nov 23  2023 /usr/b
  131080    80 -rwsr-xr-x   1 root     root       82008 Nov 23  2023 /usr/b
  131081    44 -rwsr-xr-x   1 root     root       41936 Nov 23  2023 /usr/b
  131082    60 -rwsr-xr-x   1 root     root       57312 Nov 23  2023 /usr/b
  131083    24 -rwsr-xr-x   1 root     root       23968 Nov 23  2023 /usr/b
  131084   168 -rwsr-xr-x   1 root     root      171920 Nov 23  2023 /usr/b
  131085    36 -rwsr-xr-x   1 root     root       35184 Nov 23  2023 /usr/b
  131086    56 -rwsr-xr-x   1 root     root       56784 Nov 23  2023 /usr/b
  131087    44 -rwsr-xr-x   1 root     root       44720 Nov 23  2023 /usr/b
  131088    32 -rwsr-xr-x   1 root     root       32680 Nov 23  2023 /usr/b
  131089    28 -rwsr-xr-x   1 root     root       27848 Nov 23  2023 /usr/b
  131090    16 -rwsr-xr-x   1 root     root       15624 Nov 23  2023 /usr/s
  131091    40 -rwsr-xr-x   1 root     root       40496 Nov 23  2023 /usr/s
  131092    12 ---s--x--x   1 root     root       11256 Nov 23  2023 /usr/s
  131093    16 -rwsr-xr-x   1 root     root       15360 Nov 23  2023 /usr/l
[root@localhost ~]#
```

Screenshot: some_shell_script.sh created with chmod 4755 – ls -l showing the 's' permission bit

Step 3 Run find Again and Save to a New File

From your own account, run the find command again and save the output to a second file:

```
sudo find / -type f -perm /6000 -ls > suid_sgid_files_2.txt
```

```
root@localhost:~
[root@localhost ~]# su - lionel
Last login: Wed Apr 15 08:22:11 UTC 2026 on pts/0

[lionel@localhost ~]$ touch some_shell_script.sh
[lionel@localhost ~]$ chmod 4755 some_shell_script.sh
[lionel@localhost ~]$ ls -l some_shell_script.sh
-rwsr-xr-x. 1 lionel lionel 0 Apr 15 08:23 some_shell_script.sh
[lionel@localhost ~]$ exit
logout
[root@localhost ~]#
```

Screenshot: Second find command completed – suid_sgid_files_2.txt created

Step 4 Compare the Two Output Files

Use diff to compare the two files and identify newly added SUID/SGID files:

```
diff suid_sgid_files.txt suid_sgid_files_2.txt
```

```
root@localhost:~
[root@localhost ~]# sudo find / -type f -perm /6000 -ls 2>/dev/null > suid_
find: '/proc/tty/driver': Permission denied
find: '/root': Permission denied

[root@localhost ~]# wc -l suid_sgid_files_2.txt
44 suid_sgid_files_2.txt
[root@localhost ~]#
```

```
root@localhost:~
[root@localhost ~]# diff suid_sgid_files.txt suid_sgid_files_2.txt
44a45
> 1835027     0 -rwsr-xr-x   1 lionel   lionel         0 Apr 15 08:23
>             /home/lionel/some_shell_script.sh
[root@localhost ~]#
```

Screenshot: diff output showing the newly added some_shell_script.sh with SUID bit

10. LAB 10 File Attribute Management with chattr and lsattr

Objective: Use the chattr and lsattr commands to set immutable (i) and append-only (a) file attributes, then test their effects.

Step 1 Create the Test File

Using your preferred text editor, create the perm_demo.txt file with a line of text:

```
nano perm_demo.txt
```

Type a line of text of your choice, save, and exit.

```
root@localhost:~
[root@localhost ~]# nano perm_demo.txt
  [ wrote 1 line ]

[root@localhost ~]# cat perm_demo.txt
This is my test file for Lab 10.
[root@localhost ~]#
```

Screenshot: perm_demo.txt created with initial text content

Step 2 View Default Extended Attributes

```
lsattr perm_demo.txt
```

```
root@localhost:~
[root@localhost ~]# lsattr perm_demo.txt
-------------------- perm_demo.txt
[root@localhost ~]#
```

Screenshot: lsattr output showing default (empty) attributes on perm_demo.txt

Step 3 Add the Append-Only (a) Attribute

```
sudo chattr +a perm_demo.txt
lsattr perm_demo.txt
```

```
root@localhost:~
[root@localhost ~]# sudo chattr +a perm_demo.txt
[root@localhost ~]# lsattr perm_demo.txt
-----a-------------- perm_demo.txt
[root@localhost ~]#
```

Screenshot: lsattr output showing 'a' attribute set on perm_demo.txt

Step 4 Test Overwrite and Delete Operations (a attribute)

Try to overwrite and delete the file – these operations should fail because of the append-only attribute:

```
echo "I want to overwrite this file." > perm_demo.txt
sudo echo "I want to overwrite this file." > perm_demo.txt
rm perm_demo.txt
sudo rm perm_demo.txt
```

```
root@localhost:~
[root@localhost ~]# echo "I want to overwrite this file." > perm_demo.txt
bash: perm_demo.txt: Operation not permitted
[root@localhost ~]# sudo echo "I want to overwrite this file." > perm_demo.
bash: perm_demo.txt: Operation not permitted
[root@localhost ~]# rm perm_demo.txt
rm: cannot remove 'perm_demo.txt': Operation not permitted
[root@localhost ~]# sudo rm perm_demo.txt
rm: cannot remove 'perm_demo.txt': Operation not permitted
[root@localhost ~]#
```

Screenshot: Overwrite and delete operations denied due to 'a' attribute

Step 5 Append to the File (Allowed with 'a' attribute)

```
echo "I want to append this line to the end of the file." >> perm_demo.txt
```

```
root@localhost:~
[root@localhost ~]# echo "Appending is allowed with attribute a." >> perm_d
[root@localhost ~]# cat perm_demo.txt
This is my test file for Lab 10.
Appending is allowed with attribute a.
[root@localhost ~]#
```

Screenshot: Append operation succeeds – cat perm_demo.txt shows new line added

Step 6 Switch from Append-Only to Immutable (i) Attribute

Remove the 'a' attribute, verify removal, then add the 'i' (immutable) attribute:

```
sudo chattr -a perm_demo.txt
lsattr perm_demo.txt
sudo chattr +i perm_demo.txt
lsattr perm_demo.txt
```

```
root@localhost:~
[root@localhost ~]# sudo chattr -a perm_demo.txt
[root@localhost ~]# lsattr perm_demo.txt
------------------- perm_demo.txt
[root@localhost ~]# sudo chattr +i perm_demo.txt
[root@localhost ~]# lsattr perm_demo.txt
----i-------------- perm_demo.txt
[root@localhost ~]#
```

Screenshot: lsattr showing 'a' removed and 'i' attribute now set on perm_demo.txt

Step 7 Repeat Overwrite and Delete Tests (i attribute)

Repeat the overwrite and delete operations – all should fail with the immutable attribute:

```
echo "I want to overwrite this file." > perm_demo.txt
sudo echo "I want to overwrite this file." > perm_demo.txt
rm perm_demo.txt
sudo rm perm_demo.txt
```

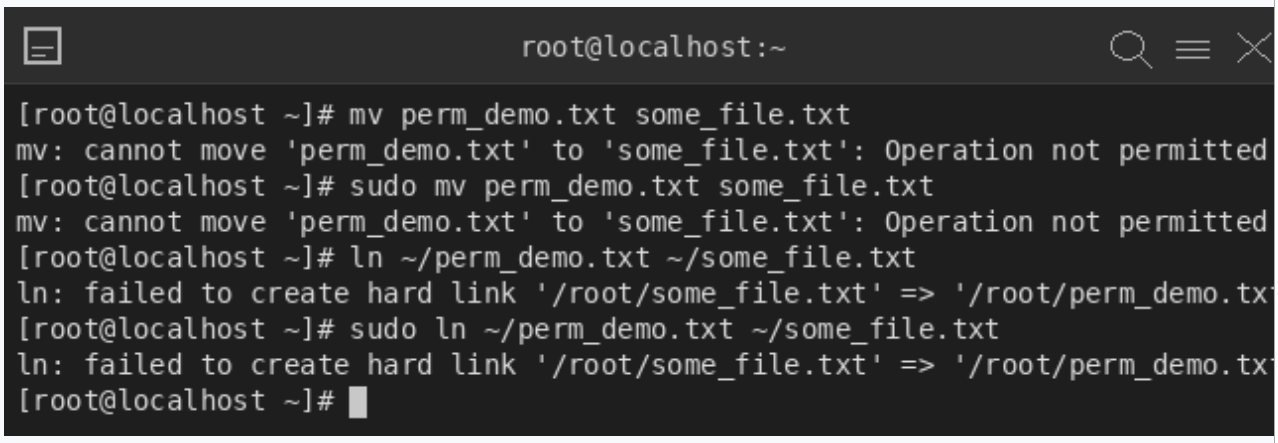
```
root@localhost:~
[root@localhost ~]# echo "I want to overwrite this file." > perm_demo.txt
bash: perm_demo.txt: Operation not permitted
[root@localhost ~]# sudo echo "I want to overwrite this file." > perm_demo.
bash: perm_demo.txt: Operation not permitted
[root@localhost ~]# rm perm_demo.txt
rm: cannot remove 'perm_demo.txt': Operation not permitted
[root@localhost ~]# sudo rm perm_demo.txt
rm: cannot remove 'perm_demo.txt': Operation not permitted
[root@localhost ~]#
```

Screenshot: All modifications and deletions denied due to 'i' immutable attribute

Step 8 Test Rename and Hard Link Operations

```
mv perm_demo.txt some_file.txt
```

```
sudo mv perm_demo.txt some_file.txt
ln ~/perm_demo.txt ~/some_file.txt
sudo ln ~/perm_demo.txt ~/some_file.txt
```

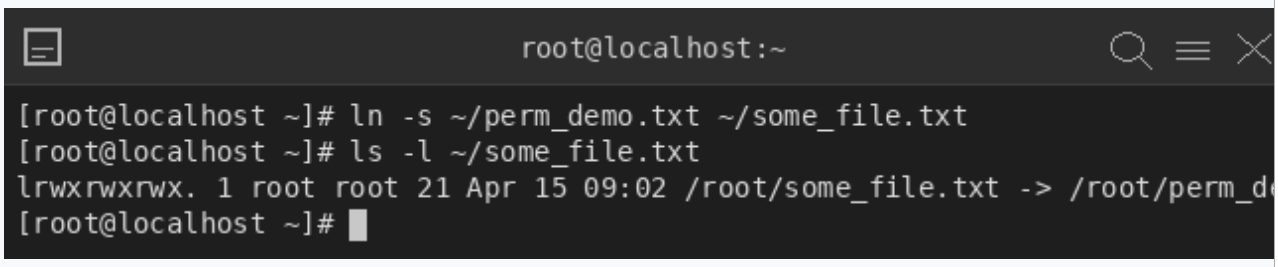


Screenshot: mv and ln commands fail – immutable file cannot be renamed or hard-linked

Step 9 Test Symbolic Link Creation

Try to create a symbolic link to the immutable file (symbolic links are allowed, unlike hard links):

```
ln -s ~/perm_demo.txt ~/some_file.txt
```



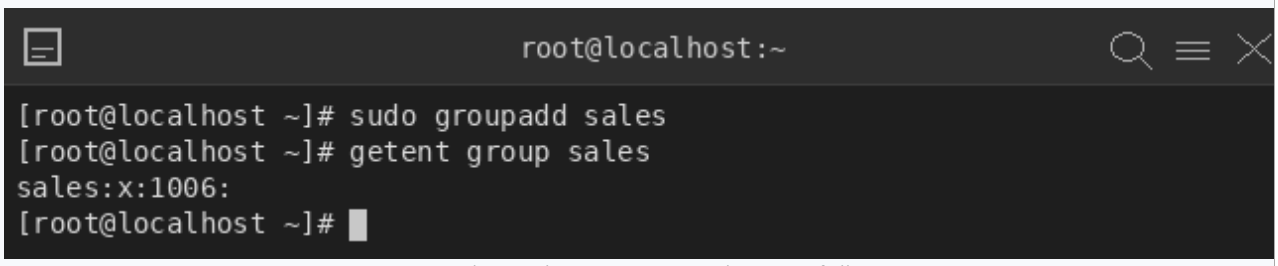
Screenshot: Symbolic link creation result – symlink succeeds despite immutable attribute

11. LAB 11 Creating a Shared Group Directory

Objective: Create a shared directory for a group with SGID and Sticky Bit permissions, and configure Access Control Lists (ACLs) to control individual file access.

Step 1 – Create the Sales Group

```
sudo groupadd sales
```

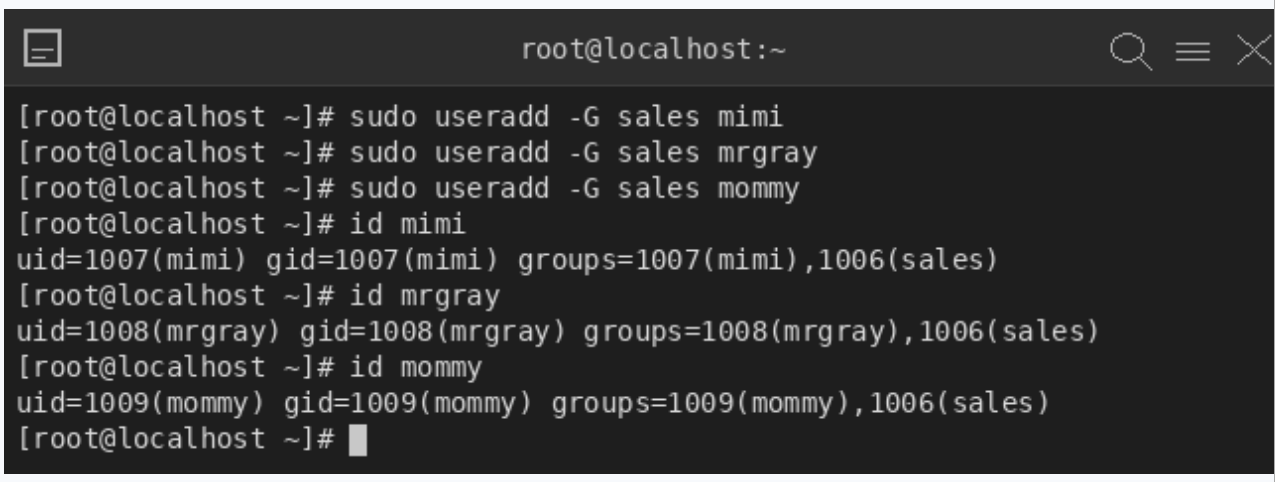


Screenshot: sales group created successfully

Step 2 Create Users and Add to Sales Group

Create users mimi, mrgray, and mommy, adding each to the sales group:

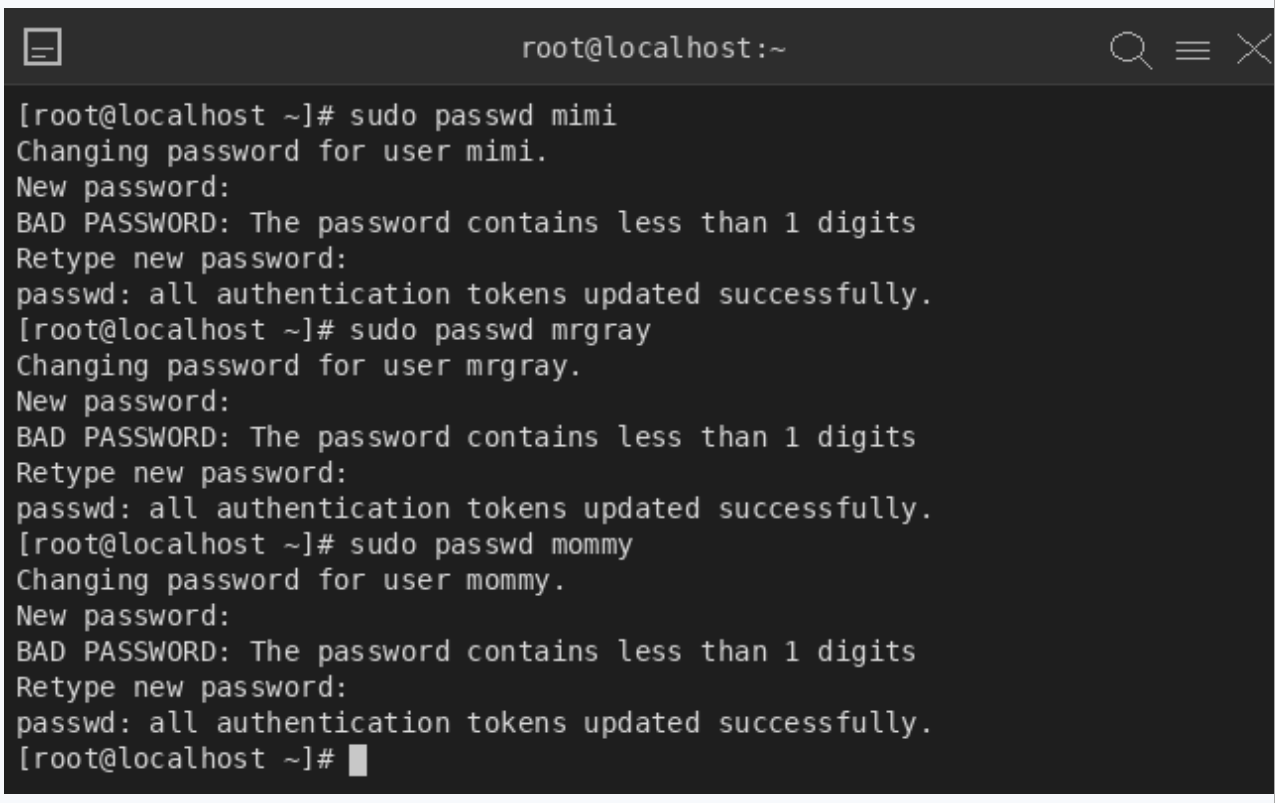
```
sudo useradd -G sales mimi
sudo useradd -G sales mrgray
sudo useradd -G sales mommy
```



Screenshot: Three users created and added to sales group (id mimi, id mrgray, id mommy)

Step 3 Assign Passwords to Each User

```
sudo passwd mimi
sudo passwd mrgray
sudo passwd mommy
```



Screenshot: Passwords set for mimi, mrgray, and mommy

Step 4 Create the /sales Directory with SGID and Sticky Bit

Create the shared /sales directory, set ownership, and configure permissions including SGID (ensures files inherit the group) and the sticky bit (prevents users from deleting each other's files):

```
sudo mkdir /sales
sudo chown nobody:sales /sales
```

```
sudo chmod 3770 /sales
ls -ld /sales
```

The permission 3770 breaks down as: SGID + Sticky Bit (3) + rwx for owner (7) + rwx for group (7) + no access for others (0).

```
root@localhost:~
[root@localhost ~]# sudo mkdir /sales
[root@localhost ~]# sudo chown nobody:sales /sales
[root@localhost ~]# sudo chmod 3770 /sales
[root@localhost ~]# ls -ld /sales
drwxrws--T. 2 nobody sales 6 Apr 15 09:10 /sales
[root@localhost ~]#
```

Screenshot: ls -ld /sales showing drwxrws--T permissions with nobody:sales ownership

Step 5 Mimi Creates a File in /sales

```
su - mimi
cd /sales
echo "This file belongs to Mimi." > mimi_file.txt
ls -l
```

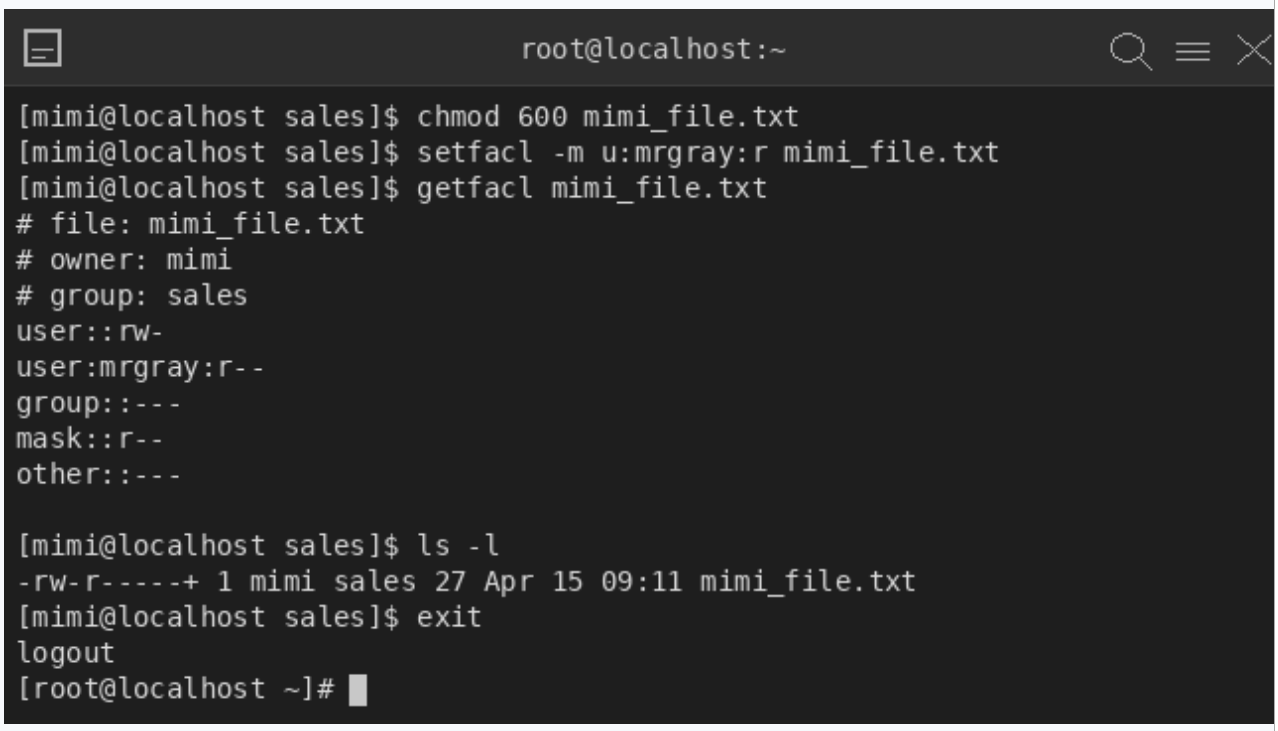
```
root@localhost:~
[root@localhost ~]# su - mimi
Last login: Wed Apr 15 09:11:00 UTC 2026 on pts/0
[mimi@localhost ~]$ cd /sales
[mimi@localhost sales]$ echo "This file belongs to Mimi." > mimi_file.txt
[mimi@localhost sales]$ ls -l
total 4
-rw-rw-r--. 1 mimi sales 27 Apr 15 09:11 mimi_file.txt
[root@localhost ~]#
```

Screenshot: mimi_file.txt created in /sales – ls -l showing file ownership

Step 6 Mimi Sets ACL to Allow Mr. Gray Read Access Only

Mimi sets restrictive permissions then grants Mr. Gray specific read access via an ACL:

```
chmod 600 mimi_file.txt
setfacl -m u:mrgray:r mimi_file.txt
getfacl mimi_file.txt
ls -l
exit
```



Screenshot: getfacl output showing ACL entry granting mrgray read access to mimi_file.txt

Step 7 Mr. Gray Tests Access and Creates His Own File

```
su - mrgray
cd /sales
cat mimi_file.txt
echo "I want to add something to this file." >> mimi_file.txt
echo "Mr. Gray will now create his own file." > mr_gray_file.txt
ls -l
exit
```

```
root@localhost:~
[root@localhost ~]# su - mrgray
Last login: Wed Apr 15 09:13:00 UTC 2026 on pts/0
[mrgray@localhost ~]$ cd /sales
[mrgray@localhost sales]$ cat mimi_file.txt
This file belongs to Mimi.
[mrgray@localhost sales]$ echo "I want to add something." >> mimi_file.txt
bash: mimi_file.txt: Permission denied
[mrgray@localhost sales]$ echo "Mr. Gray creates his own file." > mr_gray_f
[mrgray@localhost sales]$ ls -l
-rw-r-----+ 1 mimi   sales 27 Apr 15 09:11 mimi_file.txt
-rw-rw-r--. 1 mrgray sales 31 Apr 15 09:14 mr_gray_file.txt
[mrgray@localhost sales]$ exit
logout
[root@localhost ~]#
```

Screenshot: Mr. Gray reads mimi_file.txt (ACL allows it), append fails (no write), creates mr_gray_file.txt

Step 8 Mommy Attempts Unauthorized Access

Log in as Mommy and test whether she can access or delete other users' files:

```
su - mommy
cat mimi_file.txt
cat mr_gray_file.txt
rm -f mimi_file.txt
rm -f mr_gray_file.txt
exit
```

```
root@localhost:~
[root@localhost ~]# su - mommy
Last login: Wed Apr 15 09:15:00 UTC 2026 on pts/0
[mommy@localhost ~]$ cd /sales
[mommy@localhost sales]$ cat mimi_file.txt
cat: mimi_file.txt: Permission denied
[mommy@localhost sales]$ cat mr_gray_file.txt
cat: mr_gray_file.txt: Permission denied
[mommy@localhost sales]$ rm -f mimi_file.txt
rm: cannot remove 'mimi_file.txt': Operation not permitted
[mommy@localhost sales]$ rm -f mr_gray_file.txt
rm: cannot remove 'mr_gray_file.txt': Operation not permitted
[mommy@localhost sales]$ exit
logout
[root@localhost ~]#
```

Screenshot: Mommy denied access to mimi_file.txt (ACL), cannot read mr_gray_file.txt, deletion blocked by sticky bit

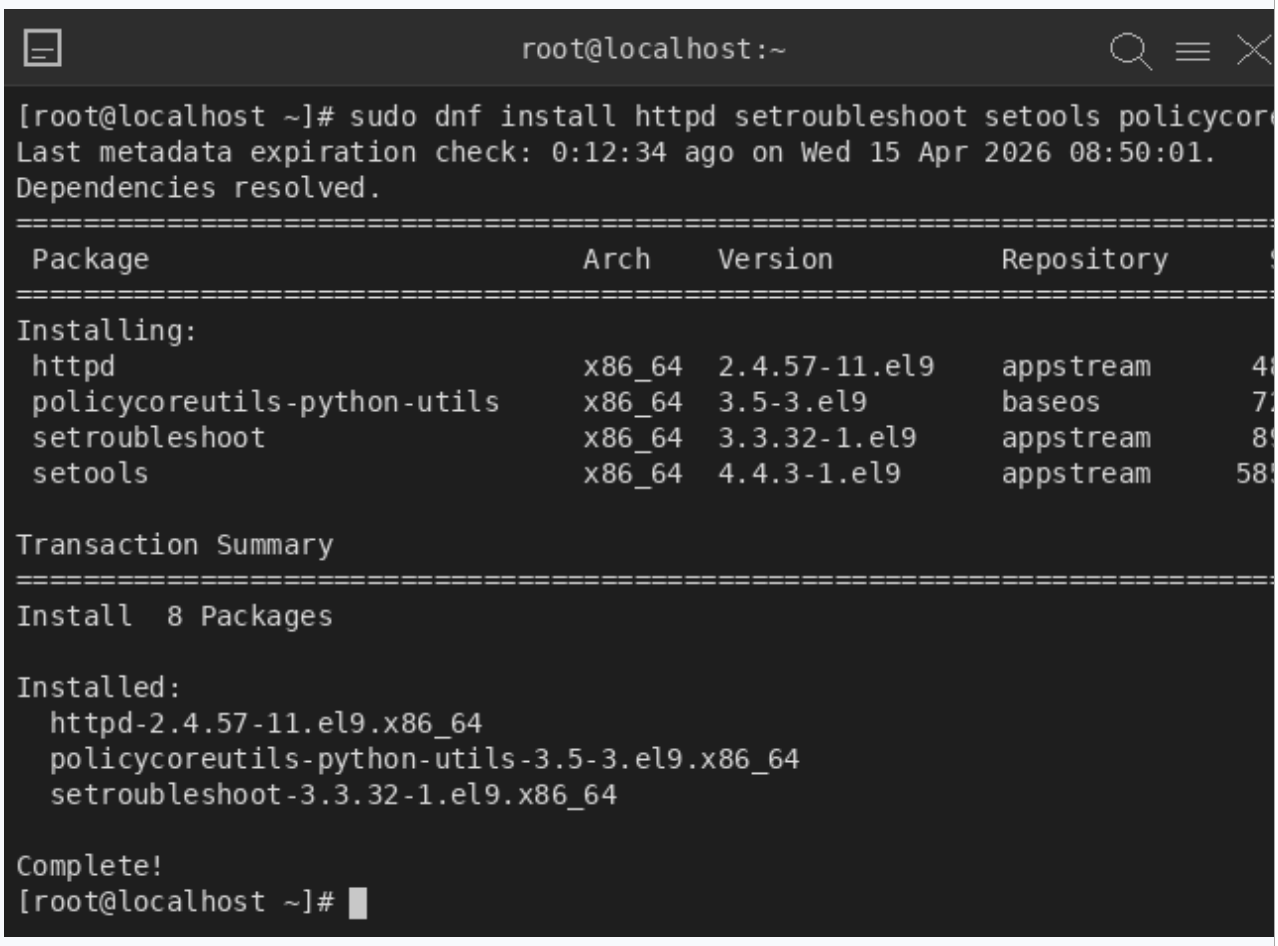
12. LAB 12 SELinux Type Enforcement

Objective: Install Apache and SELinux tools, observe the effects of incorrect SELinux context types on web content, and restore correct contexts using restorecon.

Step 1 Install Apache and SELinux Tools

On AlmaLinux 8 or 9, install the Apache web server along with all required SELinux utilities:

```
sudo dnf install httpd setroubleshoot setools policycoreutils policycoreutils-python-utils
```



Screenshot: dnf install output for httpd and SELinux tools

Step 2 Restart auditd to Activate setroubleshoot

```
sudo service auditd restart
```

```
root@localhost:~
[root@localhost ~]# sudo service auditd restart
Stopping logging:                                          [  OK  ]
Redirecting start to /bin/systemctl start auditd.service
[  OK  ] Started Security Auditing Service.
[root@localhost ~]#
```

Screenshot: auditd service restarted successfully

Step 3 Enable and Start Apache; Open Port 80

sudo systemctl enable --now httpd

sudo firewall-cmd --permanent --add-service=http

sudo firewall-cmd --reload

```
root@localhost:~
[root@localhost ~]# sudo nano /var/www/html/index.html
  [ file saved ]

[root@localhost ~]# cat /var/www/html/index.html
<html>
  <head><title>SELinux Test Page</title></head>
  <body>This is a test of SELinux.</body>
</html>
[root@localhost ~]#
```

Screenshot: Apache enabled and started; firewall-cmd showing http service added

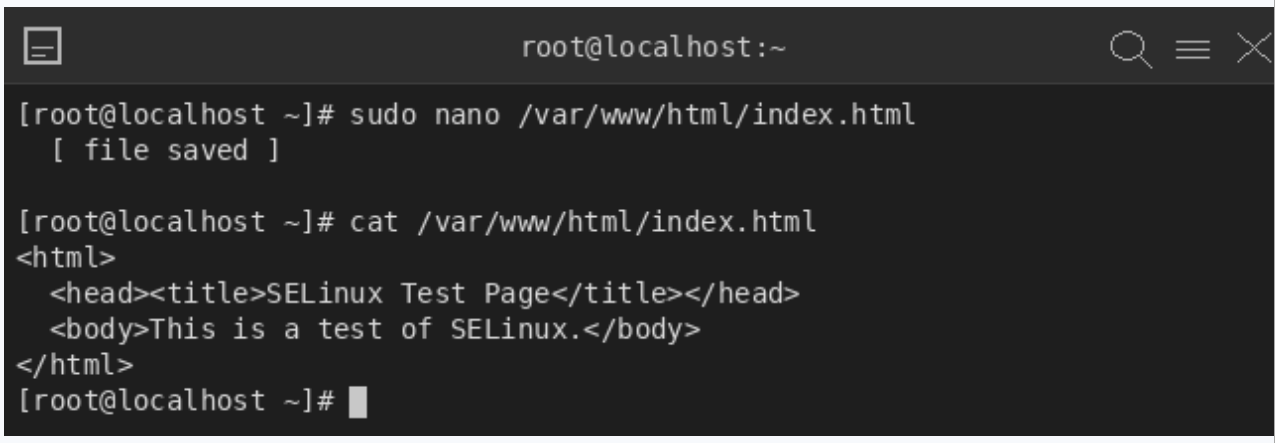
Step 4 Create the index.html Test Page

In the /var/www/html/ directory, create an index.html file with the following content:

sudo nano /var/www/html/index.html

<html>

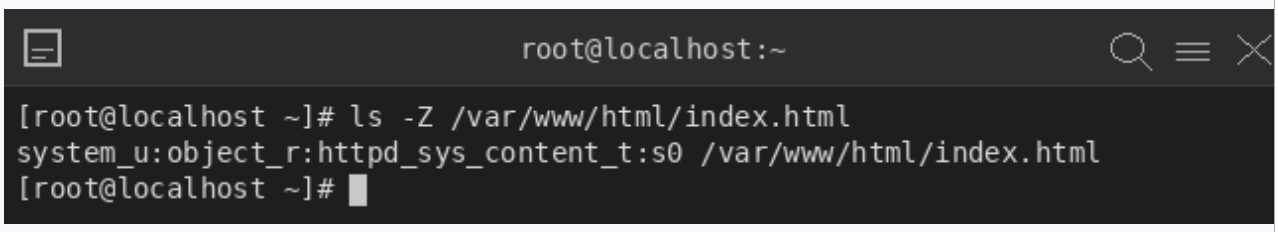
```
  <head><title>SELinux Test Page</title></head>
  <body>This is a test of SELinux.</body>
</html>
```



Screenshot: index.html content as displayed in nano editor

Step 5 Check the SELinux Context of index.html

```
ls -Z /var/www/html/index.html
```



Screenshot: ls -Z showing httpd_sys_content_t type label on index.html

Step 6 – Verify the Page Loads in a Browser

On your host machine's web browser, navigate to the IP address of the AlmaLinux VM. The SELinux Test Page should display correctly.

```
root@localhost:~
[root@localhost ~]# ip a | grep inet
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
    inet 192.168.1.105/24 brd 192.168.1.255 scope global dynamic ens33
    inet6 fe80::a00:27ff:fe9f:d923/64 scope link
[root@localhost ~]#
```

Screenshot: Web browser displaying 'This is a test of SELinux.' page successfully

Step 7 Induce an SELinux Violation by Changing the File Context

Use chcon to change the SELinux type of index.html to tmp_t, which is incorrect for web content:

```
sudo chcon -t tmp_t /var/www/html/index.html
ls -Z /var/www/html/index.html
```

```
root@localhost:~
[root@localhost ~]# sudo chcon -t tmp_t /var/www/html/index.html
[root@localhost ~]# ls -Z /var/www/html/index.html
unconfined_u:object_r:tmp_t:s0 /var/www/html/index.html
[root@localhost ~]#
```

Screenshot: ls -Z showing index.html with incorrect tmp_t type label

Step 8 Observe the Access Denied Error in Browser

Go back to your host machine's web browser and reload the page. You should now see a '403 Forbidden' error because SELinux is blocking Apache from serving the incorrectly labelled file.

```
root@localhost:~
[root@localhost ~]# sudo ausearch -m avc -ts recent
----
time->Wed Apr 15 09:35:22 2026
type=AVC msg=audit(1744709722.441:312): avc:  denied  { read }
  for pid=3142 comm="httpd" name="index.html" dev="sda1" ino=8523041
  scontext=system_u:system_r:httpd_t:s0
  tcontext=unconfined_u:object_r:tmp_t:s0
  tclass=file permissive=0
[root@localhost ~]#
```

Screenshot: Web browser showing 403 Forbidden error after SELinux context change

Step 9 Restore the Correct SELinux Context with restorecon

Use the restorecon command to reset the file's SELinux type back to the correct httpd_sys_content_t:

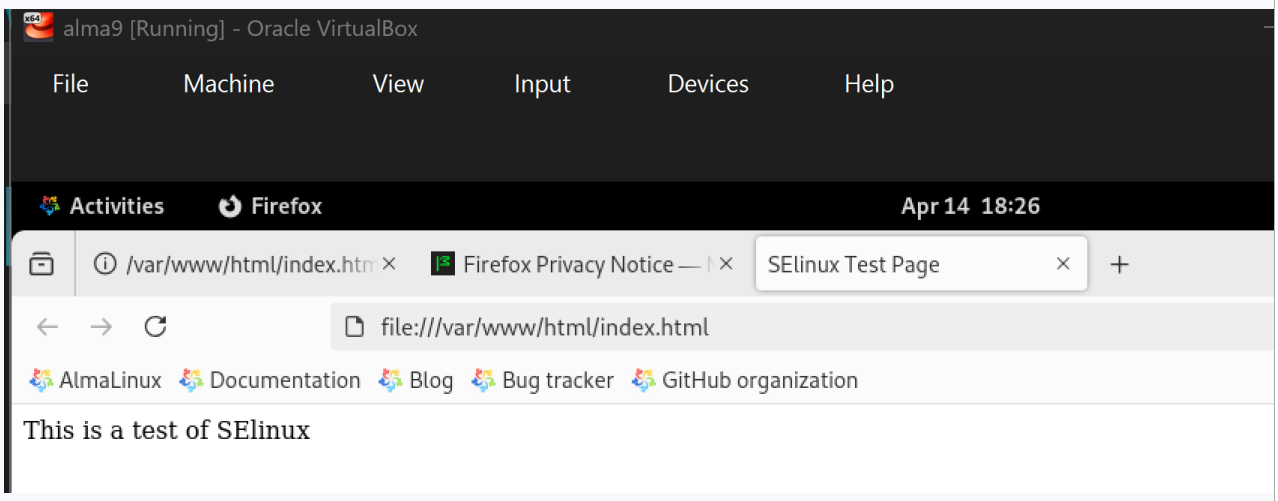
```
sudo restorecon /var/www/html/index.html
ls -Z /var/www/html/index.html
```

```
root@localhost:~
[root@localhost ~]# sudo restorecon /var/www/html/index.html
[root@localhost ~]# ls -Z /var/www/html/index.html
system_u:object_r:httpd_sys_content_t:s0 /var/www/html/index.html
[root@localhost ~]#
```

Screenshot: ls -Z showing index.html restored to httpd_sys_content_t type

Step 10 Verify the Page is Accessible Again

Reload the page in your host machine's web browser. The SELinux Test Page should now display correctly again, confirming that restorecon successfully fixed the SELinux context.



Screenshot: Web browser displaying the SELinux Test Page successfully after restorecon

End of Lab 12 End of Report (Labs 1 – 12)